

A Practical Course on Mathematical Modelling

Benders Decomposition and Column Generation in Julia

Prof. Viet Hung Nguyen

Clermont Auvergne University - Clermont-Ferrand - France

Course background

This course is based on two JuMP tutorials:

- Benders decomposition.
- Column generation: cutting stock.

The slides include the data, models, proofs, and Julia code. No other document is needed during the course.

We start from each original MIP and derive the decomposition.

The examples and numerical illustrations use the instances in these sources. Solver versions and dual choices may change iteration histories.

Part I: Benders decomposition.

- 1 Start from the network design MIP and separate its decisions.
- 2 Evaluate a candidate, construct a cut, and repeat.
- 3 Derive the cuts formally from LP duality.
- 4 Implement callbacks, model reuse, and feasibility cuts.

Part II: column generation.

- 1 Start from the cutting stock MIP and derive patterns.
- 2 Solve a small master and use its dual prices to find a new pattern.
- 3 Add columns, certify the LP optimum, and recover integer roll counts.

Goals and software

- Derive each reformulation from the original optimization model.
- Connect dual values to Benders cuts and column reduced costs.
- Implement both algorithms with JuMP and interpret their bounds.

Prerequisites: linear programming, LP duality, mixed integer programming.

Julia packages: JuMP, HiGHS, SparseArrays.

The callback variant also uses Gurobi and requires a license.

Code uses ASCII names: `theta` for θ , `pi` for π .

Network design: the original problem

Nodes 1 and 8 are the source and sink. G_{ij} is arc capacity.

- $x_{ij} = 1$ opens an arc at cost 0.1.
- $y_{ij} \geq 0$ carries flow on that arc.
- At most 11 arcs may open.

$$\begin{aligned} \min \quad & 0.1 \sum_{i,j} x_{ij} - \sum_j y_{1j} \\ \text{s.t.} \quad & \sum_{i,j} x_{ij} \leq 11, \\ & 0 \leq y_{ij} \leq G_{ij} x_{ij}, \\ & \sum_j y_{ij} = \sum_j y_{ji}, \quad i = 2, \dots, 7, \\ & x_{ij} \in \{0, 1\}. \end{aligned}$$

Network capacities

```
using JuMP
import HiGHS

G = [
    0 3 2 2 0 0 0 0
    0 0 0 0 5 1 0 0
    0 0 0 0 1 3 1 0
    0 0 0 0 0 1 0 0
    0 0 0 0 0 0 0 4
    0 0 0 0 0 0 0 2
    0 0 0 0 0 0 0 4
    0 0 0 0 0 0 0 0
]
n = size(G, 1)
```

The model uses $n \times n$ arrays. Zero capacities force zero flow.

Solving the monolithic model in JuMP

```
full = Model(HiGHS.Optimizer)
set_silent(full)
@variable(full, x[1:n, 1:n], Bin)
@variable(full, y[1:n, 1:n] >= 0)
@constraint(full, sum(x) <= 11)
@constraint(full, y .<= G .* x)
@constraint(full, [i in 2:(n-1)],
    sum(y[i, :]) == sum(y[:, i]))
@objective(full, Min, 0.1 * sum(x) - sum(y[1, :]))
optimize!(full)
assert_is_solved_and_feasible(full)
reference = objective_value(full)
```

The optimal objective is -5.1 : flow 6 and 9 open arcs.
Use -5.1 to check the result of Benders decomposition.

Fixing the openings leaves an LP

Suppose $\bar{x}$ specifies which arcs are open. The remaining decisions are the continuous flows y :

$$\begin{aligned} Q(\bar{x}) = \min_y \quad & - \sum_j y_{1j} \\ \text{s.t.} \quad & 0 \leq y_{ij} \leq G_{ij} \bar{x}_{ij}, \\ & \sum_j y_{ij} = \sum_j y_{ji}, \quad i = 2, \dots, 7. \end{aligned}$$

This is an LP. For example, $\bar{x} = 0$ forces $y = 0$ and gives $Q(0) = 0$.

For each choice of openings, this LP computes the best flow. The total cost is $0.1 \sum_{i,j} \bar{x}_{ij} + Q(\bar{x})$.

Separating the two stages

The master retains the opening decisions and their budget:

$$\min 0.1 \sum_{i,j} x_{ij} + \theta, \quad \sum_{i,j} x_{ij} \leq 11, \quad x_{ij} \in \{0, 1\}.$$

For fixed x^k , the subproblem computes the best flow:

$$Q(x^k) = \min_y \left\{ - \sum_j y_{1j} : 0 \leq y \leq G \circ x^k, \text{ flow conservation} \right\}.$$

Zero flow is feasible for every binary x^k in this original model.

Initialize $M = - \sum_{i,j} G_{ij} = -29$. This loose lower bound is valid because flow cannot exceed capacities.

Estimating the flow cost in the master

The master chooses x , but it does not contain the flow variables y . It needs an estimate of their best contribution $Q(x)$.

Introduce θ to represent this contribution:

$$\min 0.1 \sum_{i,j} x_{ij} + \theta.$$

Initially, require only $\theta \geq -29$, a valid lower bound. The master may then underestimate the cost of its chosen network.

The second-stage LP evaluates that network. A Benders cut prevents the master from repeating the same incorrect cost estimate.

Constructing the master

```
master = Model(HiGHS.Optimizer)
set_silent(master)
@variable(master, x[1:n, 1:n], Bin)
@variable(master, theta >= -sum(G))
@constraint(master, sum(x) <= 11)
@objective(master, Min, 0.1 * sum(x) + theta)
```

Initially there are no cuts. The master underestimates the flow cost.

Its first solution opens no arcs and sets $\theta = -29$.

The subproblem then finds $Q(0) = 0$ and corrects that estimate.

Continuous copies of the opening decisions

The second stage needs the values of the master decisions, not new integer decisions.

Represent those values through a continuous copy $\tilde{x}$ and impose

$$\tilde{x} = \bar{x}.$$

The LP still optimizes y . Its dual multipliers for these equalities describe how the optimal value changes with the supplied first-stage decision.

Role	Mathematics	Julia name
Master opening decisions	x	<code>x</code>
Numerical candidate	$\bar{x}$	<code>xbar</code>
Continuous copy in the LP	$\tilde{x}$	<code>x_copy</code>
Flow decisions in the LP	y	<code>y</code>

Building the LP at xbar

```
function recourse(xbar)
    sub = Model(HiGHS.Optimizer)
    set_silent(sub)
    @variable(sub, x_copy[i=1:n, j=1:n] == xbar[i, j])
    @variable(sub, y[1:n, 1:n] >= 0)
    @constraint(sub, y .<= G .* x_copy)
    @constraint(sub, [i in 2:(n-1)],
        sum(y[i, :]) == sum(y[:, i]))
    @objective(sub, Min, -sum(y[1, :]))
```

sub is the JuMP model of the second-stage LP.

The equality in the declaration fixes each $x_copy[i, j]$ to its numerical value $xbar[i, j]$.

Only the flows remain to be chosen.

The function continues on the next slide.

What the LP returns

```
optimize!(sub)
    assert_is_solved_and_feasible(sub; dual = true)
    return (q = objective_value(sub),
           y = value.(y),
           pi = reduced_cost.(x_copy))
end
```

The returned object groups three results under explicit names:

Julia field	Mathematical meaning	Use
ret.q	$Q(\bar{x})$	Actual second-stage cost
ret.y	Optimal flow matrix	Feasible network solution
ret.pi	Multipliers π of $\tilde{x} = \bar{x}$	Cut coefficients

The point in `value.(y)` and `reduced_cost.(x_copy)` applies the function separately to every variable in the array.

Constructing a cut from the LP solution

At a candidate x^k , the LP returns $Q(x^k)$ and multipliers π^k . LP duality gives a lower estimate valid at every decision with feasible recourse:

$$Q(x) \geq Q(x^k) + (\pi^k)^\top (x - x^k).$$

Require the master's estimate to satisfy the same inequality:

$$\theta \geq Q(x^k) + (\pi^k)^\top (x - x^k).$$

At $x = x^k$, this forces $\theta \geq Q(x^k)$. The next master solve includes this constraint.

The next slides show the first cut and derive it from the dual.

Interpreting the first cut

At $x^1 = 0$, the second-stage optimum is $Q(0) = 0$.

One optimal dual solution gives

$$\theta \geq -3x_{12} - 2x_{13} - 2x_{14}.$$

Only arcs leaving the source appear in this particular cut.

Opening those arcs can improve the flow objective, but later cuts also account for bottlenecks downstream.

Dual degeneracy may produce different valid cuts on another run.

The network subproblem and its explicit dual

Let B be the node–arc incidence matrix for intermediate nodes, and let $e_{ij} = 1$ if $i = 1$, and 0 otherwise.

$$\begin{aligned} Q(x) = \min_y \quad & -e^\top y \\ \text{s.t.} \quad & By = 0, \quad 0 \leq y \leq G \circ x. \end{aligned}$$

With capacity multipliers $\mu \leq 0$ and free conservation multipliers η , the dual is

$$\begin{aligned} \max_{\mu, \eta} \quad & \sum_{i,j} G_{ij} x_{ij} \mu_{ij} \\ \text{s.t.} \quad & B^\top \eta + \mu \leq -e, \quad \mu \leq 0. \end{aligned}$$

Only the dual objective depends on x .

Deriving the actual network cut

An optimal dual solution (μ^k, η^k) at x^k yields

$$\theta \geq \sum_{i,j} G_{ij} \mu_{ij}^k x_{ij}.$$

The dual multipliers of $\tilde{x}_{ij} = x_{ij}^k$ satisfy $\pi_{ij}^k = G_{ij} \mu_{ij}^k$ and

$$Q(x^k) = \sum_{i,j} \pi_{ij}^k x_{ij}^k.$$

The affine optimality cut has zero constant term for this network.

At $x^k = 0$, one valid dual optimum has $\mu_{12}^k = \mu_{13}^k = \mu_{14}^k = -1$ and zero capacity multipliers elsewhere. Therefore

$$\theta \geq -3x_{12} - 2x_{13} - 2x_{14}.$$

Other dual optima can yield different valid cuts.

A canonical formulation for deriving cuts

$$\begin{array}{ll}\min_{x,y} & c^\top x + d^\top y \\ \text{s.t.} & Ax + By \geq b, \\ & x \in X \subseteq \mathbb{Z}^m, \quad y \geq 0.\end{array}$$

X contains the first-stage constraints. Equalities can be written as pairs of inequalities and free variables as differences of nonnegative variables.

For fixed x , define

$$Q(x) = \min\{d^\top y : By \geq b - Ax, y \geq 0\}.$$

Assume the dual feasible set defined next is nonempty. Then feasible recourse has a finite optimum.

The dual of the subproblem

The dual of the second-stage LP is

$$Q(x) = \max_{u \in D} u^\top (b - Ax), \quad D = \{u \geq 0 : B^\top u \leq d\},$$

whenever the primal is feasible, by strong duality.

Changing x changes the dual objective, but does not change D . Therefore, for every $u \in D$ and every x with feasible recourse,

$$u^\top (b - Ax) \leq Q(x).$$

This weak-duality inequality is the source of every optimality cut.

Deriving an optimality cut

At a master candidate (x^k, θ^k) , solve the LP and its dual:

$$u^k \in \arg \max_{u \in D} u^\top (b - Ax^k), \quad Q(x^k) = (u^k)^\top (b - Ax^k).$$

Weak duality gives the valid master inequality

$$\theta \geq (u^k)^\top (b - Ax).$$

Set $\pi^k = -A^\top u^k$. Expanding around x^k gives

$$(u^k)^\top (b - Ax) = Q(x^k) + (\pi^k)^\top (x - x^k).$$

Its violation is $Q(x^k) - \theta^k$. Add it when this exceeds the cut tolerance.

An optimal extreme point of D suffices. Any optimal dual solution also gives a valid cut that is tight at x^k .

The restricted master after k cuts

For the canonical model, the current master is

$$\begin{aligned} \min_{x, \theta} \quad & c^\top x + \theta \\ \text{s.t.} \quad & x \in X, \quad \theta \geq M, \\ & \theta \geq Q(x^\ell) + (\pi^\ell)^\top (x - x^\ell), \quad \ell = 1, \dots, k. \end{aligned}$$

Each cut comes from an evaluated candidate. M is a valid initial lower bound.

Solving this model supplies the next x to evaluate and a lower bound on the original objective.
Evaluating that x supplies a feasible upper bound.

Bounds and termination

An optimally solved master gives a lower bound L_k .

A feasible second-stage solution gives $U_k = c^\top x^k + Q(x^k)$.

$$L_k \leq z^* \leq U_k, \quad \text{UB}_k = \min_{\ell \leq k} U_\ell.$$

A stopping rule that also handles $\text{UB}_k = 0$ is

$$\text{UB}_k - L_k \leq \varepsilon \max\{1, |\text{UB}_k|\}.$$

Keep the best feasible upper bound across iterations. The scaled gap remains defined when that bound is zero.

If the master stops early, use its solver objective bound for the lower bound.

Evaluating a candidate and updating bounds

```
best_ub = Inf
best_x = nothing
for k in 1:100
    optimize!(master)
    assert_is_solved_and_feasible(master; allow_local = false)
    lb = objective_value(master)
    xk = value.(x)
    ret = recourse(xk)
    candidate_ub = 0.1 * sum(xk) + ret.q
    if candidate_ub < best_ub
        global best_ub = candidate_ub
        global best_x = copy(xk)
    end
end
```

The loop continues on the next slide.

Store the incumbent decision together with its upper bound.

Stopping or adding the next cut

```
gap = (best_ub - lb) / max(1.0, abs(best_ub))
println((iteration = k, lower = lb,
        upper = best_ub, gap = gap))
if gap <= 1e-6
    break
end
@constraint(master,
    theta >= ret.q + sum(ret.pi .* (x .- xk)))
end
# Check the incumbent after the loop:
final_y = recourse(best_x).y
```

An iteration limit alone does not certify optimality.
Compare the final bounds and the monolithic objective.

Reading the cut expression in Julia

```
@constraint(master,  
    theta >= ret.q + sum(ret.pi .* (x .- xk)))
```

This is the expression

$$\theta \geq Q(x^k) + \sum_{i,j} \pi_{ij}^k (x_{ij} - x_{ij}^k).$$

- `x` contains master variables. `xk` contains numerical values.
- `x .- xk` subtracts the values entry by entry.
- `ret.pi .* (...)` multiplies matching entries.
- `sum(...)` adds the resulting affine expressions.

The cut is a linear constraint in the master variables x and θ .

An illustrative Benders iteration history

Iteration	Master lower bound	Candidate upper bound
1	−29.0	0.0
2	−6.7	0.3
3	−6.5	0.5
4	−6.2	−4.2
5	−6.1	−4.1
6	−6.1	−4.1
7	−5.1	−4.1
8	−5.1	−5.1

Candidate upper bounds can worsen between iterations. Keeping the best feasible bound retains −4.2 at iterations 5–7.

Adding cuts inside branch and bound

The iterative method restarts a master solve after each new cut.

A lazy constraint callback evaluates integer candidates during one solve.

- 1 Read the candidate x^k and its estimate θ^k .
- 2 Solve the second-stage LP at x^k .
- 3 If $\theta^k < Q(x^k)$, submit the violated optimality cut.

The callback master uses Gurobi and the subproblem uses HiGHS. HiGHS does not support this lazy constraint callback.

Use callback accessors to read candidates while the solver is running.

The callback master

```
import Gurobi

lazy_master = Model(Gurobi.Optimizer)
set_silent(lazy_master)
@variable(lazy_master, x[1:n, 1:n], Bin)
@variable(lazy_master, theta >= -sum(G))
@constraint(lazy_master, sum(x) <= 11)
@objective(lazy_master, Min, 0.1 * sum(x) + theta)
```

The variables belong to `lazy_master`.

The callback on the next slide uses these x and θ references.

Submitting a violated Benders cut

```
function benders_callback(cb)
    status = callback_node_status(cb, lazy_master)
    status == MOI.CALLBACK_NODE_STATUS_INTEGER || return
    xk = callback_value(cb, x)
    tk = callback_value(cb, theta)
    ret = recourse(xk)
    if tk < ret.q - 1e-6
        cut = @build_constraint(
            theta >= ret.q + sum(ret.pi .* (x .- xk)))
        MOI.submit(lazy_master, MOI.LazyConstraint(cb), cut)
    end
    return
end

set_attribute(lazy_master, MOI.LazyConstraintCallback(),
              benders_callback)

optimize!(lazy_master)
assert_is_solved_and_feasible(lazy_master)
```

Reusing the subproblem

In `recourse(xbar)`, every call constructs a new LP. The constraints and objective remain unchanged between calls.

Only the supplied opening values change. We can therefore:

- 1 Build the LP once, with continuous variables `x_copy` and `y`.
- 2 Fix `x_copy` to the current candidate.
- 3 Solve the LP and read its objective and dual multipliers.
- 4 Replace the fixed values when the next candidate arrives.

To do this inside a function, we must retrieve the variables stored in `sub`.

Constructing the subproblem once

```
sub = Model(HiGHS.Optimizer)
set_silent(sub)
@variable(sub, x_copy[1:n, 1:n])
@variable(sub, y[1:n, 1:n] >= 0)
@constraint(sub, y .<= G .* x_copy)
@constraint(sub, [i in 2:(n-1)],
    sum(y[i, :]) == sum(y[:, i]))
@objective(sub, Min, -sum(y[1, :]))
```

Keep the model and variable references in memory.

Solve the LP after fixing x_copy to the master candidate x^k .

Reading variables with `sub[:x_copy]`

```
@variable(sub, x_copy[1:n, 1:n])  
# The variables are also registered in the model:  
sub[:x_copy]
```

`sub` is the model object. JuMP stores its named variables in a dictionary.

`:x_copy` is a Julia symbol used as a key. It represents the name `x_copy`, rather than the contents of a Julia variable.

`sub[:x_copy]` retrieves the array of JuMP variable references. It does not return their numerical values.

Use this access inside a function that receives `sub`.

Fixing the copied variables

```
fix.(sub[:x_copy], xbar)
```

`fix(variable, value)` sets one variable to one numerical value. The point applies it to every matching pair of array entries.

For matrices, the same operation can be written:

```
for i in axes(xbar, 1), j in axes(xbar, 2)
    fix(sub[:x_copy][i, j], xbar[i, j])
end
```

It imposes $\tilde{x}_{ij} = \bar{x}_{ij}$ for all i, j . The flow variables y remain optimization variables. A later call replaces the fixed values with the next candidate.

Updating fixed values in place

```
function recourse_inplace(sub, xbar)
    fix.(sub[:x_copy], xbar)
    optimize!(sub)
    assert_is_solved_and_feasible(sub; dual = true)
    return (q = objective_value(sub),
            y = value.(sub[:y]),
            pi = reduced_cost.(sub[:x_copy]))
end
# In the iterative loop, replace:
# ret = recourse(xk)
# with:
# ret = recourse_inplace(sub, xk)
```

Rebuild a fresh master before running this variant.

Reuse can reduce model construction work and enable LP reoptimization.

When the second stage becomes infeasible

Add a positive total-flow requirement:

$$\sum_{i,j} y_{ij} \geq 1.$$

With $x = 0$, capacities force $y = 0$, so the LP is infeasible.

This constraint concerns total flow over all arcs. A unit traversing several arcs contributes several times to the sum.

There is no finite second-stage optimum at this candidate. A dual infeasibility certificate supplies a feasibility cut instead.

An infeasible candidate needs a different cut

With the new requirement $\sum_{i,j} y_{ij} \geq 1$, the candidate $x^k = 0$ has no feasible second-stage flow.

There is no optimal second-stage cost or optimal dual point to return. The master must learn that this choice of openings is infeasible.

The certificate gives a linear inequality that:

- holds for every opening decision that allows a feasible flow,
- fails at the current candidate x^k .

We call this inequality a feasibility cut.

Deriving a feasibility cut

If $By \geq b - Ax^k$, $y \geq 0$ is infeasible, Farkas' lemma supplies

$$r^k \geq 0, \quad B^\top r^k \leq 0, \quad (r^k)^\top (b - Ax^k) > 0.$$

For any feasible recourse decision x and its feasible y ,

$$(r^k)^\top (b - Ax) \leq (r^k)^\top By \leq 0.$$

Hence the valid feasibility cut is

$$(r^k)^\top (b - Ax) \leq 0.$$

The strict positive value at x^k proves that the cut excludes the candidate. It contains no θ because it restricts the domain of Q .

The complete Benders master

Let $\mathcal{V}$ be the extreme points of D and $\mathcal{R}$ its extreme rays, with recession cone

$$\text{rec}(D) = \{r \geq 0 : B^\top r \leq 0\}.$$

With $D \neq \emptyset$, the original problem is equivalent to

$$\begin{aligned} \min_{x, \theta} \quad & c^\top x + \theta \\ \text{s.t.} \quad & x \in X, \\ & \theta \geq u^\top (b - Ax), \quad u \in \mathcal{V}, \\ & r^\top (b - Ax) \leq 0, \quad r \in \mathcal{R}. \end{aligned}$$

The ray constraints ensure that the subproblem is feasible. The point constraints ensure that $\theta \geq Q(x)$.

Benders generates only the inequalities needed during optimization.

Connecting the dual derivation to JuMP

Fix the continuous copy $\tilde{x} = x^k$. The associated dual multipliers provide a subgradient of Q at x^k :

$$\pi^k = -A^\top u^k.$$

The same optimality cut can therefore be written

$$\theta \geq Q(x^k) + (\pi^k)^\top (x - x^k).$$

For an infeasibility certificate, define

$$v^k = (r^k)^\top (b - Ax^k), \quad \rho^k = -A^\top r^k.$$

The feasibility cut becomes

$$v^k + (\rho^k)^\top (x - x^k) \leq 0.$$

At optimality, `reduced_cost.(x_copy)` returns the dual multipliers π^k .

Code: `pi` denotes π^k and `rho` denotes ρ^k .

Solving and checking the dual certificate

```
# Add to the reusable second-stage model:
@constraint(sub, sum(y) >= 1)

function recourse_with_feasibility(sub, xbar)
    fix.(sub[:x_copy], xbar)
    optimize!(sub)
    if is_solved_and_feasible(sub; dual = true)
        return (feasible = true, q = objective_value(sub),
                pi = reduced_cost.(sub[:x_copy]))
    elseif dual_status(sub) != MOI.INFEASIBILITY_CERTIFICATE
        error("No dual infeasibility certificate available")
    end
    return (feasible = false, v = dual_objective_value(sub),
            rho = reduced_cost.(sub[:x_copy]))
end
```

Read ray information only after checking the dual status.

Two branches in the Benders loop

```
ret = recourse_with_feasibility(sub, xk)
if ret.feasible
    candidate_ub = 0.1 * sum(xk) + ret.q
    # Update the incumbent and check the gap as before.
    @constraint(master,
        theta >= ret.q + sum(ret.pi .* (x .- xk)))
else
    @constraint(master,
        ret.v + sum(ret.rho .* (x .- xk)) <= 0)
end
```

An optimality cut requires θ to satisfy a new lower bound on $Q(x)$.

A feasibility cut excludes a candidate with no feasible flow.

Rebuild the master for this modified problem. An infeasible candidate provides no new feasible upper bound.

Why the feasibility cut removes the candidate

With continuous copies fixed to the candidate, a certificate gives

$$v^k + (\rho^k)^\top (x - x^k) \leq 0.$$

At $x = x^k$, the left-hand side equals $v^k > 0$. The current candidate therefore violates the cut.

For every choice with feasible recourse, the certificate inequality holds.

In the code, `ret.rho` is the vector $\rho^k = -A^\top r^k$.

Check the solver's certificate status before constructing this cut.

The cut-generation procedure

- 1 Solve the restricted master and obtain (x^k, θ^k) .
- 2 Solve the second-stage LP at x^k .
- 3 If it is feasible, obtain an optimal dual point u^k . Update the feasible upper bound and add

$$\theta \geq (u^k)^\top (b - Ax)$$

if $Q(x^k) - \theta^k > \varepsilon_{\text{cut}}$.

- 4 If it is infeasible, obtain a certificate r^k and add

$$(r^k)^\top (b - Ax) \leq 0.$$

- 5 Repeat until the bounds meet the stopping tolerance.

If the solver provides no certificate, stop: no feasibility cut is available.

Cutting stock starts with rolls and pieces

A supplier has large rolls of width $W = 100$.

Customers request d_i pieces of width w_i , for $i = 1, \dots, 20$.

Decisions:

- Which large rolls should we use?
- How many pieces of each type should we cut from each roll?

Minimize the number of large rolls while meeting demand.

Start by modeling each potential roll and its piece counts.

Cutting stock instance

i	w_i	d_i	i	w_i	d_i
1	75.0	38	11	30.8	44
2	70.0	41	12	29.8	49
3	68.4	34	13	20.1	37
4	65.5	23	14	16.2	36
5	59.6	18	15	14.5	42
6	53.8	33	16	11.0	33
7	53.0	36	17	8.6	47
8	51.0	41	18	8.2	35
9	50.2	35	19	6.6	49
10	32.2	37	20	5.1	42

All examples in this part use these data and $W = 100$. The demand constraints allow overproduction.

Loading the cutting stock data in Julia

```
using JuMP
import HiGHS
import SparseArrays

W = 100.0
w = [75.0, 70.0, 68.4, 65.5, 59.6, 53.8, 53.0,
     51.0, 50.2, 32.2, 30.8, 29.8, 20.1, 16.2,
     14.5, 11.0, 8.6, 8.2, 6.6, 5.1]
d = [38, 41, 34, 23, 18, 33, 36, 41, 35, 37,
     44, 49, 37, 36, 42, 33, 47, 35, 49, 42]
I = length(w)
J = 1_000
```

The vectors w and d store piece widths and demands. Use these data throughout the cutting stock implementations.

Original MIP: decisions for each roll

For each potential roll $j = 1, \dots, J$:

$x_{ij} \in \mathbb{Z}_+$: pieces of type i cut from roll j ,

$y_j \in \{0, 1\}$: whether roll j is used.

$$\begin{aligned} \min \quad & \sum_{j=1}^J y_j \\ \text{s.t.} \quad & \sum_{i=1}^I w_i x_{ij} \leq W y_j, \quad j = 1, \dots, J, \\ & \sum_{j=1}^J x_{ij} \geq d_i, \quad i = 1, \dots, I. \end{aligned}$$

If $y_j = 0$, all x_{ij} on roll j must be zero.

The original MIP in JuMP

```
roll_model = Model(HiGHS.Optimizer)
set_silent(roll_model)
@variable(roll_model, pieces[1:I, 1:J] >= 0, Int)
@variable(roll_model, used[1:J], Bin)
@objective(roll_model, Min, sum(used))
@constraint(roll_model, [i in 1:I],
             sum(pieces[i, j] for j in 1:J) >= d[i])
@constraint(roll_model, [j in 1:J],
             sum(w[i] * pieces[i, j] for i in 1:I) <= W * used[j])
set_time_limit_sec(roll_model, 5.0)
optimize!(roll_model)
solution_summary(roll_model)
```

A time limit may return an incumbent and a bound without proving optimality.
Check `has_values` before reading the incumbent objective.

Why reconsider this formulation?

For $I = 20$ and $J = 1000$, the model contains 20,000 integer piece counts and 1000 binary decisions.

Many equivalent solutions differ only by a permutation of roll indices.

In the LP relaxation, roll use y_j and piece counts x_{ij} may be fractional. A physical cutting plan requires integer rolls and pieces.

For each used roll, the piece counts satisfy:

$$x_{\cdot j} \in \mathbb{Z}_+^I, \quad w^\top x_{\cdot j} \leq W \quad \text{when } y_j = 1.$$

Group rolls with identical contents.

A pattern is a feasible column of the original MIP

Look at one used roll: its column is

$$a = (x_{1j}, \dots, x_{lj})^\top, \quad a \in \mathbb{Z}_+^l, \quad w^\top a \leq W.$$

Call this vector a cutting pattern. It describes one roll's contents.

Examples for this instance:

- One type 1 and one type 13: $75 + 20.1 = 95.1 \leq 100$.
- Nineteen type 20 pieces: $19(5.1) = 96.9 \leq 100$.
- One type 1 and one type 2: $75 + 70 = 145 > 100$, infeasible.

The capacity constraint defines which patterns fit in a roll.

Grouping identical rolls

Suppose a feasible cutting plan contains these used rolls:

Roll	Contents	Occupied width
1	One type 1 and one type 13	$75 + 20.1 = 95.1$
2	One type 1 and one type 13	$75 + 20.1 = 95.1$
3	Nineteen type 20 pieces	$19(5.1) = 96.9$

The first two rolls have the same pattern. Describe this part of the plan as

$$\lambda_p = 2 \quad \text{for } a_p = e_1 + e_{13}, \quad \lambda_q = 1 \quad \text{for } a_q = 19e_{20}.$$

Both descriptions use three rolls and produce exactly the same pieces. Pattern counts replace the labels of individual rolls.

From labeled rolls to pattern counts

Let $\mathcal{P}$ contain all nonempty feasible patterns. Let a_{ip} count pieces of type i in pattern p .

Group rolls that use the same composition. Let λ_p be their count.

$$\begin{aligned} \min \quad & \sum_{p \in \mathcal{P}} \lambda_p \\ \text{s.t.} \quad & \sum_{p \in \mathcal{P}} a_{ip} \lambda_p \geq d_i, \quad i = 1, \dots, I, \\ & \lambda_p \in \mathbb{Z}_+, \quad p \in \mathcal{P}. \end{aligned}$$

The demand rows sum production across pattern counts.

One unit of λ_p costs one roll.

Why the integer reformulation is equivalent

From the original MIP: each used roll supplies a feasible pattern. Grouping identical columns gives integer counts λ with the same cost.

From pattern counts: create λ_p separate rolls with contents a_p . These rolls satisfy the original capacity and demand constraints.

This correspondence requires enough potential rolls in the original model. Here the homogeneous-pattern solution uses $386 < 1000$ rolls, so an optimal solution fits within $J = 1000$.

The formulation removes roll labels, but $\mathcal{P}$ can be very large.

Why generate columns instead of enumerating patterns?

The full pattern LP has one variable for every feasible integer vector

$$a \in \mathbb{Z}_+^I, \quad 0 < w^\top a \leq W.$$

Enumerating all such vectors can dominate the computation.

Start with a small feasible subset $\mathcal{P}_0$. Solve its LP relaxation, then ask whether an omitted pattern can improve it.

Each new pattern adds one column to the demand matrix.

The pricing problem will answer that question without listing all patterns.

Starting with homogeneous patterns

For each type i , fill one roll with as many pieces of that type as possible:

$$a^{(i)} = \lfloor W/w_i \rfloor e_i.$$

These I patterns make the restricted master feasible because each demand type has a column that supplies it.

Examples:

$$a^{(1)} = e_1, \quad a^{(10)} = 3e_{10}, \quad a^{(20)} = 19e_{20}.$$

There are no mixed compositions at initialization.

Representing patterns in Julia

```
# A homogeneous pattern for type i:  
i = 20  
patterns_i = SparseArrays.sparsevec(  
    [i], [floor{Int, W / w[i]}], I)
```

A pattern has I entries, one count per piece type.

The arguments of `sparsevec` give the nonzero indices, their counts, and the full vector length. Unlisted entries are zero.

For type 20, the vector has length 20, entry 20 equals 19, and entries 1–19 equal zero.

After storing all patterns in `patterns`, `patterns[p][i]` means a_{ip} , the count of type i in pattern p .

Building the initial integer master

```
patterns = [SparseArrays.sparsevec(
    [i], [floor{Int, W / w[i]}], I) for i in 1:I]

rmp = Model{HiGHS.Optimizer}
set_silent(rmp)
@variable(rmp, lambda[1:length(patterns)] >= 0, Int)
@objective(rmp, Min, sum(lambda))
@constraint(rmp, demand[i in 1:I],
    sum(patterns[p][i] * lambda[p]
        for p in eachindex(patterns)) >= d[i])
optimize!(rmp)
assert_is_solved_and_feasible(rmp)
```

The explicit coefficient $a_{ip} = \text{patterns}[p][i]$ also works when mixed patterns are added.

What the initial integer solution tells us

With only homogeneous patterns, each type is independent:

$$\lambda_i = \left\lceil \frac{d_i}{\lfloor W/w_i \rfloor} \right\rceil.$$

The initial integer master needs 386 rolls.

This plan meets demand. Mixing piece types can use the remaining width and reduce the number of rolls.

Remove integrality from the master to obtain LP dual prices.

Relaxing the restricted master

```
unset_integer(lambda)
optimize!(rmp)
assert_is_solved_and_feasible(rmp; dual = true)
pi = dual.(demand)
```

The restricted LP is

$$\min \sum_{p \in \mathcal{P}_k} \lambda_p, \quad A_k \lambda \geq d, \quad \lambda \geq 0.$$

For this minimization model with $\geq$ demand rows, the dual values satisfy $\pi_i \geq 0$.

Only LP duals drive pricing. The integer master does not provide them.

The dual of the restricted master

For a current pattern subset $\mathcal{P}_k$:

$$\begin{aligned} \max_{\pi} \quad & \sum_i d_i \pi_i \\ \text{s.t.} \quad & \sum_i a_{ip} \pi_i \leq 1, \quad p \in \mathcal{P}_k, \\ & \pi_i \geq 0. \end{aligned}$$

π_i is the current marginal value of supplying one more piece of type i .

Each existing pattern has dual value at most its cost of one roll.

An omitted pattern might violate that condition.

Deriving reduced cost from a missing column

A potential pattern a has objective coefficient 1 and demand column a . Its reduced cost is

$$\bar{c}(a) = 1 - \pi^\top a.$$

If $\pi^\top a > 1$, then $\bar{c}(a) < 0$. This pattern violates the dual constraint $\pi^\top a \leq 1$.

Search for the feasible pattern with largest dual value:

$$\max_{a \in \mathbb{Z}_+^I} \pi^\top a \quad \text{s.t.} \quad w^\top a \leq W.$$

This is an integer knapsack with unlimited copies of each piece type.

Computing the reduced cost of a new pattern

Initially, pattern i contains $k_i = \lfloor W/w_i \rfloor$ copies of type i . The restricted LP has separate rows $k_i \lambda_i \geq d_i$. Its optimal dual prices are $\pi_i = 1/k_i$.

For types 9, 13, and 17:

$$k_9 = 1, \quad k_{13} = 4, \quad k_{17} = 11, \quad \pi_9 = 1, \quad \pi_{13} = \frac{1}{4}, \quad \pi_{17} = \frac{1}{11}.$$

Consider $a_9 = 1$, $a_{13} = 2$, $a_{17} = 1$:

$$w^\top a = 99, \quad \pi^\top a = 1 + \frac{2}{4} + \frac{1}{11} \approx 1.591.$$

Its reduced cost is $1 - 1.591 \approx -0.591$. It is feasible and violates a missing dual constraint, so add this pattern.

The pricing problem in JuMP

```
function price_pattern(w, W, pi)
    pricing = Model(HiGHS.Optimizer)
    set_silent(pricing)
    @variable(pricing, count[1:length(w)] >= 0, Int)
    @constraint(pricing, sum(w .* count) <= W)
    @objective(pricing, Max, sum(pi .* count))
    optimize!(pricing)
    assert_is_solved_and_feasible(pricing; allow_local = false)
    if objective_value(pricing) <= 1.0 + 1e-8
        return nothing
    end
    a = round.(Int, value.(count))
    return SparseArrays.sparse(a)
end
```

Pricing is an integer problem even though the master is an LP.
An exact pricing optimum is needed to justify termination.

Solving the master and asking for a new column

```
while true
  optimize!(rmp)
  assert_is_solved_and_feasible(rmp; dual = true)
  pi = dual.(demand)
  a = price_pattern(w, W, pi)
  if a === nothing
    break
  end
end
```

The loop continues on the next slide.

Recompute the dual values after each master solve. The next pricing problem uses the updated prices.

Adding the column and returning to the master

```
push!(patterns, a)
    new_var = @variable(rmp, lower_bound = 0.0)
    push!(lambda, new_var)
    set_objective_coefficient(rmp, new_var, 1.0)
    for i in eachindex(d)
        set_normalized_coefficient(demand[i], new_var, a[i])
    end
end
lp_value = objective_value(rmp)
```

Each new variable counts rolls using the new pattern.

Keep these variables continuous until column generation ends.

Adding a pattern to the demand rows

Before adding a , demand row i is

$$\sum_{p \in \mathcal{P}_k} a_{ip} \lambda_p \geq d_i.$$

Create a continuous variable $\lambda_{\text{new}} \geq 0$ with cost 1. The same row becomes

$$\sum_{p \in \mathcal{P}_k} a_{ip} \lambda_p + a_i \lambda_{\text{new}} \geq d_i.$$

- `push!(lambda, new_var)` stores the new variable reference.
- `set_objective_coefficient(..., 1.0)` assigns its one-roll cost.
- `set_normalized_coefficient(demand[i], new_var, a[i])` assigns its contribution to demand row i .

Solve the updated LP before reading new dual prices.

What the pricing stopping test proves

If exact pricing finds

$$\max\{\pi^\top a : a \in \mathbb{Z}_+^I, w^\top a \leq W\} \leq 1,$$

then every possible column satisfies its dual constraint.

The restricted primal solution is feasible for the full LP by assigning zero to all omitted variables. The current dual is also feasible for the full LP.

Strong duality therefore certifies the full pattern LP optimum.

$$z_{\text{LP}}^* \leq z_{\text{IP}}^*.$$

The code uses 10^{-8} as the pricing tolerance. This gives a numerical stopping certificate subject to solver tolerances.

Reading the fractional solution

```
for p in eachindex(lambda)
    rolls = value(lambda[p])
    if rolls > 1e-6
        println((pattern = p, rolls = rolls))
    end
end
rounded_rolls = sum(ceil{Int, value(v)} for v in lambda)
```

Fractional roll counts solve the relaxation, but cannot be cut physically.
Rounding every count upward preserves demand because all coefficients are nonnegative.
Illustrative output: 33 generated patterns and 306 rolls after upward rounding.

Reintroducing integrality

```
set_integer.(lambda)
optimize!(rmp)
assert_is_solved_and_feasible(rmp)
integer_rolls = objective_value(rmp)

production = [sum(patterns[p][i] * value(lambda[p]))
               for p in eachindex(patterns)) for i in 1:I]
@assert all(production .>= d .- 1e-6)
```

For the illustrative pattern set, the restricted integer optimum is 299 rolls.

This improves on upward rounding, which uses 306 rolls. Different generated pattern sets may give different integer solutions.

What is certified for the integer problem?

Let z_{RIP} be the integer solution over generated patterns. After exact LP pricing termination:

$$\lceil z_{\text{LP}}^* \rceil \leq z_{\text{IP}}^* \leq z_{\text{RIP}}.$$

Equality of the outer bounds certifies global integer optimality.

Otherwise, the restricted integer solve gives a feasible upper bound. Omitted patterns may still improve the integer optimum.

Branch and price generates columns within branch and bound to pursue an exact integer solution.